Greedy methods

Can choosing the best immediate candidate guarantee a globally optimal solution? The key is proving that the choice can still extend to an optimum.

Study **activity selection → Huffman coding → matroids and job scheduling**, moving from concrete exchanges to reusable structural conditions.

Prerequisites are sorting, priority queues, trees, sets, and induction. Distinguish the local rule, feasibility, and optimality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Turn a choice rule into a proof

Design question	Required evidence
What is a legal solution?	Fix compatibility, prefix-code, or independence constraints.
Which candidate comes next?	Define the candidates, ordering, and tie handling.
Why can an optimum survive?	Give an exchange, contraction, or matroid argument.
Does a changed objective preserve the rule?	Test weighted variants and changed constraints with counterexamples.

Activity selection, Huffman coding, and matroids require their own proofs. A plausible local rule alone proves nothing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Finish early to preserve later choices

One room serves activities with fixed times and equal value. Select as many mutually compatible activities as possible.

At each step, choose the compatible activity with the earliest finish time. An exchange argument proves this choice safe. Success on one example is insufficient.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Touching endpoints are compatible

Activity i occupies the half-open interval $[s_i,f_i)$, with $s_i < f_i$.

Two activities are compatible exactly when $f_i \leq s_j$ or $f_j \leq s_i$. If the selected prefix ends at L, accept a candidate when $s_i \geq L$.

[0,3) and [3,5) are compatible. [0,3) and [2,5) overlap. A required cleanup time δ changes the test to $s_i \geq L+\delta$.

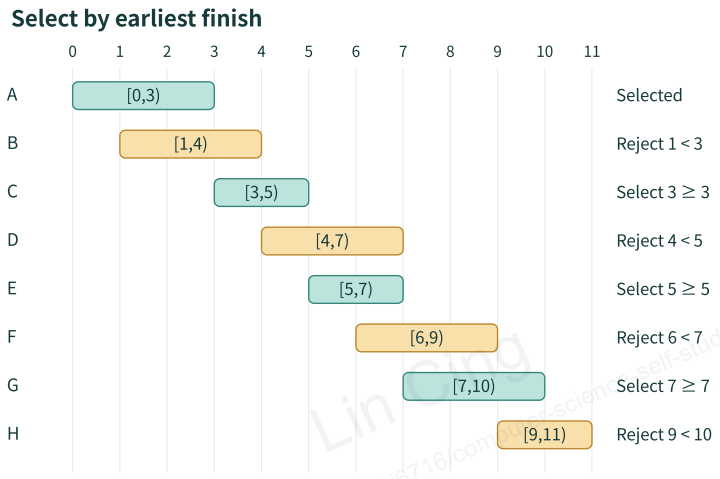
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sort by finish time, then scan once

```
selected = []
last_finish = −∞
for (id, start, finish) in sort_by_finish(activities):
    if start >= last_finish:
        append id to selected
        last_finish = finish
return selected
```

Break finish-time ties in any fixed order. This example uses ID order. Empty input returns an empty set. Rejection leaves last_finish unchanged.

A timeline of eight activities



Time (hours). Teal selected; amber rejected for overlap.

Times are in hours. Input A[0,3), B[1,4), C[3,5), D[4,7), E[5,7), F[6,9), G[7,10), H[9,11). Teal marks selected A, C, E, G. Amber marks a conflict with the most recently selected activity.

Compare each start against the last finish

Candidate	L before	Test	Action	L after
A [0,3)	$-\infty$	$0 \geq -\infty$	Select A	3
B [1,4)	3	$1 < 3$	Reject B	3
C [3,5)	3	$3 \geq 3$	Select C	5
D [4,7)	5	$4 < 5$	Reject D	5
E [5,7)	5	$5 \geq 5$	Select E	7
F [6,9)	7	$6 < 7$	Reject F	7
G [7,10)	7	$7 \geq 7$	Select G	10
H [9,11)	10	$9 < 10$	Reject H	10

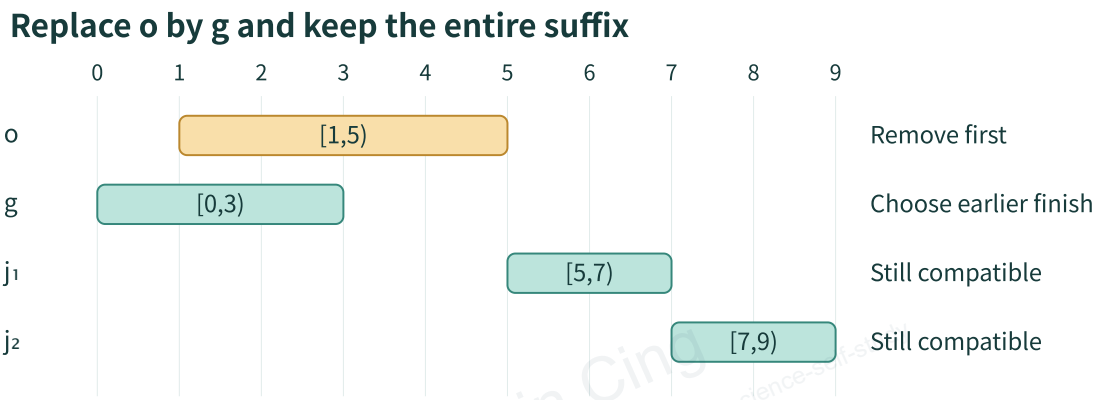
The output has four activities. The optimum need not be unique. A, C, E, H is another set of size four.

Earliest start and shortest duration can fail

Rule	Counterexample intervals	Rule obtains	Better solution
Earliest start	X[0,8), A[1,3), B[3,5), C[5,7)	X, size 1	A, B, C, size 3
Shortest duration	A[0,4), X[3,5), B[4,9)	X, size 1	A, B, size 2

The shortest-duration rule repeatedly selects a shortest activity compatible with all previous selections. X has duration 2, strictly less than A or B, but crosses their shared endpoint. A failed earliest-start rule does not refute earliest finish.

Exchange the first activity without losing count



$f_g = 3 \leq f_o = 5 \leq s_{j_1} = 5$

Consider any nonempty residual problem. Every candidate starts at or after the current boundary L. Let g be a candidate with minimum finish time. Let O be an optimal solution and o its first activity. Then $f_g \leq f_o$.

The exchanged solution stays feasible and optimal

If g=o, O already makes the greedy choice. Otherwise define $O' = (O - \{o\}) \cup \{g\}$.

1. Activity g starts at or after L, so it respects the fixed prefix.
2. Every remaining activity j in O satisfies $s_j \geq f_o \geq f_g$, so replacing o by g preserves compatibility.
3. Activity g cannot already occur later in O. Positive duration and compatibility would imply $f_g > f_o$, contradicting $f_g \leq f_o$.
4. Therefore $|O'| = |O|$. The solution O' is optimal and starts with g.

The conclusion asserts that some optimal solution contains g. It does not assert that every optimal solution contains g.

Optimal suffixes and induction prove the whole algorithm

After selecting g, retain R={i | $s_i \geq f_g$}. No candidate can precede g, because every candidate finishes at or after $f_g > s_g$.

If an optimal solution containing g had a nonoptimal suffix for R, replacing that suffix with a larger one would improve the original solution. Hence OPT(S)=1+OPT(R).

The empty set is the base case. The exchange argument makes the first choice safe. Induction on the smaller residual set proves the complete greedy solution optimal. Optimal substructure alone does not establish a safe greedy choice.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Cost and assumptions of the theorem

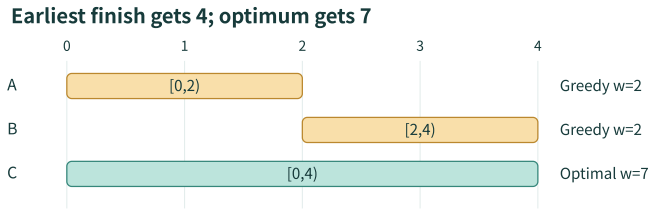
Sorting costs O(n log n). Scanning costs Θ(n). The total is O(n log n), or Θ(n) when input is already sorted by finish time.

The scan uses O(1) auxiliary space beyond sorting and output. The output contains at most n IDs.

The proof assumes one resource, fixed positive-duration intervals, and a cardinality objective. Scanning backward and selecting the latest compatible start is also correct by a symmetric exchange proof. This differs from scanning forward by earliest start.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Weights break the value-preserving exchange



Time (hours). Choose C, exclude A and B because 7 > 2+2.

A[0,2) has weight 2, B[2,4) has weight 2, and C[0,4) has weight 7. Earliest finish selects A and B for value 4. The optimum selects C for value 7.

An exchange preserves cardinality but may reduce total weight. This disproves the original rule for the weighted version. It does not prove that every conceivable greedy rule fails.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Weighted intervals require a take-or-skip recurrence

Sort by finish time. Let p(j) be the largest i<j with $f_i \leq s_j$, or 0 if none exists. Let M(j) be the maximum weight among the first j activities.

M(0)=0, and M(j)=max(M(j−1), w_j+M(p(j))).

Skipping j leaves the first j−1 activities. Taking j leaves a compatible prefix among the first p(j). These cases cover every optimal solution.

For order A, B, C, p=(0,1,0) and M=(0,2,4,7). Traceback returns {C}. Binary search computes predecessors. Total time is O(n log n), with O(n) DP and traceback time and O(n) space.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Turn symbol frequencies into a minimum bit count

Huffman optimizes binary, symbol-by-symbol, prefix-free codes over a finite alphabet. The input is a nonnegative weight w_c per symbol. The output assigns a codeword to every symbol. The main proof starts with $n \geq 2$ and positive weights; boundary cases follow.

Symbol	a	b	c	d	e	f
Frequency, thousands	45	13	12	16	9	5

A fixed-length code uses 3 bits per occurrence. Huffman uses 2.24 bits on average, or 224 thousand bits, excluding the codebook and framing.

The original scan, page 20, identifies Huffman as greedy. CLRS supplies the construction and proof. The two MIT editions supply the general greedy-choice and optimal-substructure framework.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prefix-free codes support immediate decoding

Prefix-free means that no codeword is a prefix of another codeword. In a binary tree, symbols occupy leaves only. A left edge is 0; a right edge is 1.

For example, $\{0, 10, 110, 111\}$ is prefix-free; $\{0, 01\}$ is not. On reaching a leaf, output its symbol and restart at the root. No separator is needed.

Distinct codewords may still be ambiguous. With $\{0, 01, 10\}$, the string 010 can mean 0|10 or 01|0. Prefix-free is sufficient for unique decoding, but is not necessary.

The optimization domain here is symbol-by-symbol prefix-free coding. It does not compare against block compressors that exploit dependencies between symbols.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kraft constrains codeword lengths

For a finite binary prefix-free code with nonnegative integer lengths ℓ_i,

$$\sum_i 2^{-\ell_i} \leq 1.$$

A node at depth ℓ corresponds to a binary interval of length $2^{-\ell}$ in $[0, 1)$. Prefix-free words have disjoint intervals, so their total length is at most 1.

The converse also holds. Sort the lengths in increasing order and assign the leftmost unused dyadic interval at each length. Previously assigned shorter intervals occupy whole depth-ℓ cells. The total-length bound guarantees a free cell, giving a prefix-free code.

Lengths 1,2,3,3 sum to 1; lengths 1,1,2 sum to 1.25 and are impossible. For $n \geq 2$, lengths must be at least 1. Length 0 allows only one symbol. This criterion cannot be applied to arbitrary codes whose only requirement is distinct codewords.

Weighted external path length is the objective

For a code tree T, leaf depth $d_T(c)$ equals codeword length. Minimize

$$B(T) = \sum_c w_c d_T(c).$$

With occurrence counts, B is measured in bits. With probabilities, it is bits per symbol. With positive weights, deleting a unary internal node strictly lowers the cost of its descendant leaves. Thus an optimal tree must be full.

Each internal node weighs the sum of its descendant leaves. A leaf contributes once for each ancestor, hence

$$B(T) = \sum_{v \text{ internal}} w(v) = \text{sum of all merge weights}.$$

These are two ways to check the same cost. The root weight is only $\sum_c w_c$, not the total coding cost.

Extract the two lightest trees at every round

A queue item is a whole subtree, keyed by its total weight. A merge replaces two items by one, giving exactly $n - 1$ rounds.

```
Q ← min-priority queue of symbol leaves
while |Q| > 1:
    x ← extract-min(Q)
    y ← extract-min(Q)
    z ← Node(weight=x.weight+y.weight, left=x, right=y)
    insert(Q,z)
return extract-min(Q)
```

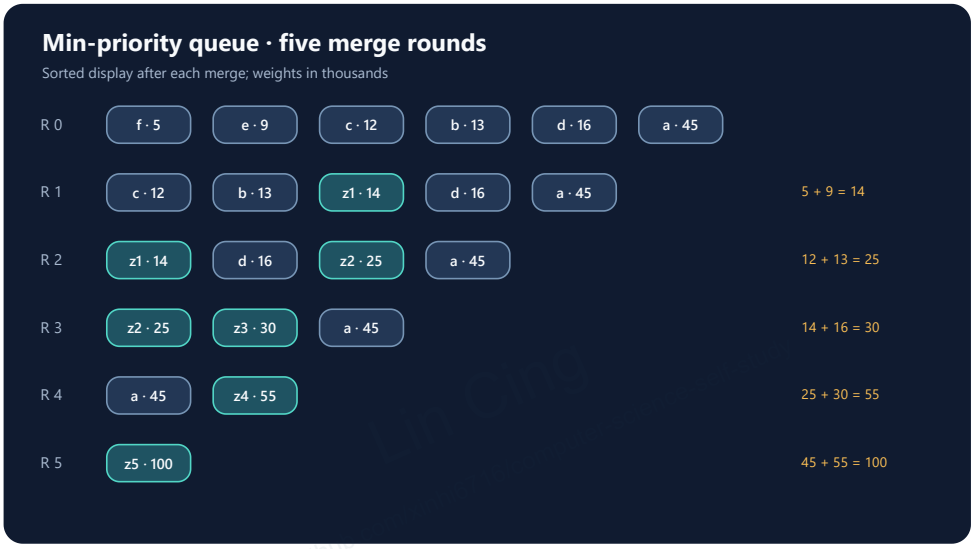
The queue subtrees partition the original alphabet. Their weights always sum to 100. Reinsert each new subtree; scanning only the original frequencies does not implement Huffman.

Audit all five queue rounds

The displayed queue is sorted for readability. A heap itself need not be fully sorted. Node z_i is created in round i.

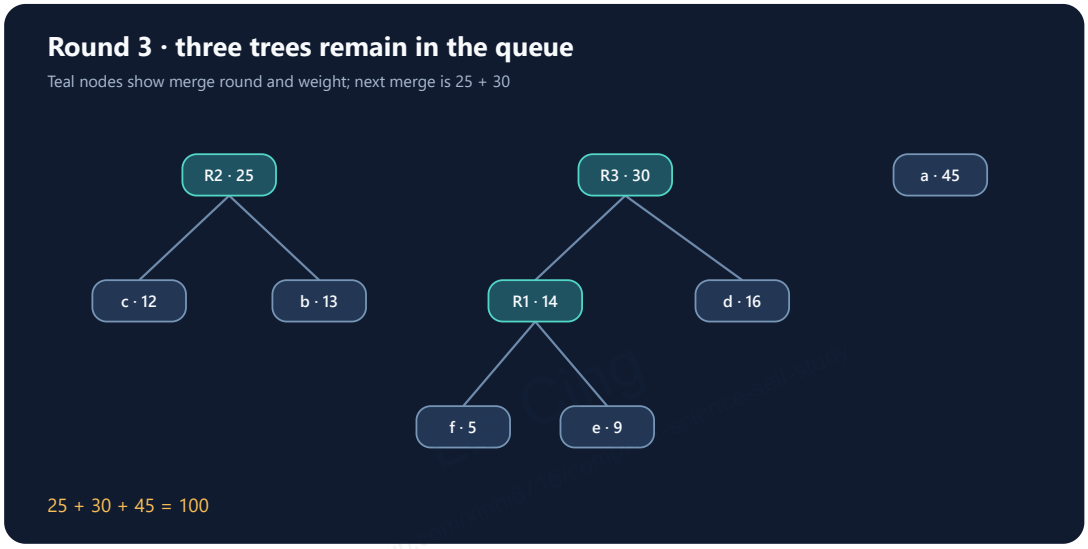
Round	Extract	Insert	Queue after merge, sorted by weight
0			f 5,e 9,c 12,b 13,d 16,a 45
1	f 5,e 9	$z_1$14	c 12,b 13,$z_1$14,d 16,a 45
2	c 12,b 13	$z_2$25	$z_1$14,d 16,$z_2$25,a 45
3	$z_1$14,d 16	$z_3$30	$z_2$25,$z_3$30,a 45
4	$z_2$25,$z_3$30	$z_4$55	a 45,$z_4$55
5	a 45,$z_4$55	$z_5$100	$z_5$100

Audit all five queue rounds (continued)

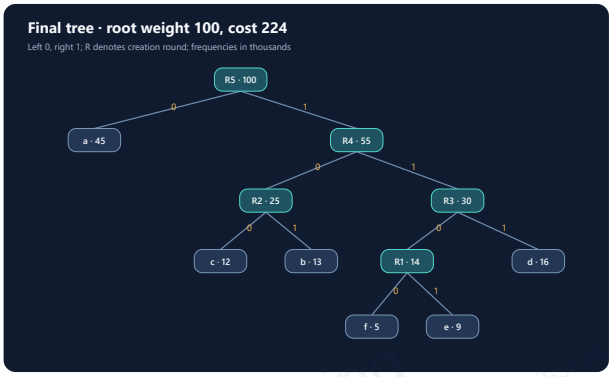


After round 3, the queue contains three trees of weights 25,30,45. The next round merges 25 and 30; it does not attach the new node 30 directly to 45.

The intermediate forest is algorithm state



Read codewords from the root and check both costs



Symbol	a	b	c	d	e	f
Codeword	0	101	100	111	1101	1100
Depth	1	3	3	3	4	4

Merge weights 14,25,30 already contribute to the final cost. Moving a subtree up by one level adds one bit to every descendant codeword, costing exactly that subtree weight.

$B = 45 + 39 + 36 + 48 + 36 + 20 = 224$ thousand bits.

The merge sum is $14 + 25 + 30 + 55 + 100 = 224$. The root weighs 100 thousand occurrences. The Kraft sum is $1/2 + 3/8 + 2/16 = 1$. The message abc encodes as 0101100.

Exchange establishes an optimal tree compatible with the choice

Let x, y be the two minimum-weight symbols. In an optimal full binary tree, choose deepest sibling leaf positions a, b at depth D. A deepest leaf has a leaf sibling; an internal sibling would have a still deeper descendant.

First exchange x with a. If x occupies b, swap the two sibling positions first at no cost. The exchange changes cost by

$$B(T_{new}) - B(T) = (w_a - w_x)(d_T(x) - D) \leq 0.$$

Now exchange y with the leaf in position b. Symbol x is fixed at a; y has minimum weight among all remaining leaves, so the same inequality applies. Skip any exchange whose symbol is already in position.

Neither exchange increases cost. Since the original tree is optimal, the new tree is optimal too. Thus some optimal tree has x, y as deepest siblings. We do not claim that every optimal tree has this shape.

Contraction and induction prove global optimality

Contract sibling leaves x, y into a leaf z of weight $w_z = w_x + w_y$. Other depths stay unchanged; both leaves lose one edge.

$$B(T) = B(T') + w_x + w_y.$$

If a contracted optimal tree T' were suboptimal, replace it by a cheaper reduced tree and expand z. Expansion adds the same fixed $w_x + w_y$, contradicting optimality of the original tree.

Induct from one leaf, whose tree cost is 0. Exchange guarantees an optimal solution containing this round's minimum siblings. The induction hypothesis gives an optimal Huffman solution for the reduced $n - 1$ symbols. Expansion restores a fixed cost and therefore gives an optimal solution for the original alphabet.

A cheapest immediate merge alone is not a proof. The proof needs both the cost-preserving exchange and the optimal reduced problem.

Ties, one symbol, and zero weights

Break equal-weight ties arbitrarily. Swapping left and right does not change lengths or cost. A fixed creation sequence can be a secondary key for reproducible codebooks.

For one symbol, the tree model assigns the empty word ϵ, costing 0. A format requiring a nonempty code may instead assign 0, costing w. That adds a different length constraint. Decoding an empty word requires an external symbol count. Return an empty result for an empty alphabet.

Zero-weight symbols still need codewords if they belong to the alphabet. Huffman and the non-increasing exchange proof remain valid. Removing a unary node may now leave the cost unchanged, so not every optimal tree must be full. For a of weight 1 and b of weight 0, codes 00,01 cost 2 while 0,1 cost 1. If all weights are zero, every valid code costs 0.

Weights must be finite and nonnegative. Do not normalize by a zero total weight.

Construction complexity and reproducible checks

Build a binary min-heap in $O(n)$. Each of the $n-1$ rounds uses two extract-min operations and one insertion, each $O(\log n)$. Tree construction therefore takes $O(n \log n)$ time and $O(n)$ node space, with $2n-1$ nodes.

If the weights are already sorted, keep a leaf queue and a merge-node queue and choose from their two fronts. Tree construction then takes $O(n)$; sorting is a separate cost.

These bounds cover tree construction. Materializing every codeword string also costs $\Theta(\sum_c \ell_c)$ time and output space, which can be $\Theta(n^2)$. Encoding a message additionally processes the emitted bits.

Check that no listed codeword prefixes another. Compute both costs: 45+39+36+48+36+20=224 and 14+25+30+55+100=224. The exchange and contraction proof establishes optimality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

When does taking the heaviest element work?

Activity selection has its own earliest-finish rule. A matroid instead guarantees sorted greedy optimization for every nonnegative weight assignment.

We express feasibility as independent sets, prove the exchange step, and turn missed-deadline penalties into saved weight.

The main source is CLRS § § 16.4–16.5. Undirected forests illustrate set operations. MST algorithms and the cut property belong to Chapter 8.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A finite ground set and two axioms

Let M=(E,$\mathcal{I}$), where E is a finite ground set, $\mathcal{I}\subseteq 2^E$, and $\varnothing\in\mathcal{I}$.

- Heredity. If $A\in\mathcal{I}$ and $B\subseteq A$, then $B\in\mathcal{I}$.
- Exchange. If $A,B\in\mathcal{I}$ and $|A|<|B|$, some $x\in B\setminus A$ satisfies $A\cup\{x\}\in\mathcal{I}$.

These conditions define a matroid. Heredity alone defines an independence system. Exchange must hold for every eligible pair, not merely one successful example.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Maximal and maximum have different meanings

A maximal independent set admits no further element. A maximum independent set has the largest cardinality.

In a matroid, if a maximal set A were smaller than an independent set B, exchange would supply an element to add to A. This contradicts maximality. All maximal independent sets therefore have the same size. They are bases, and their size is the rank r.

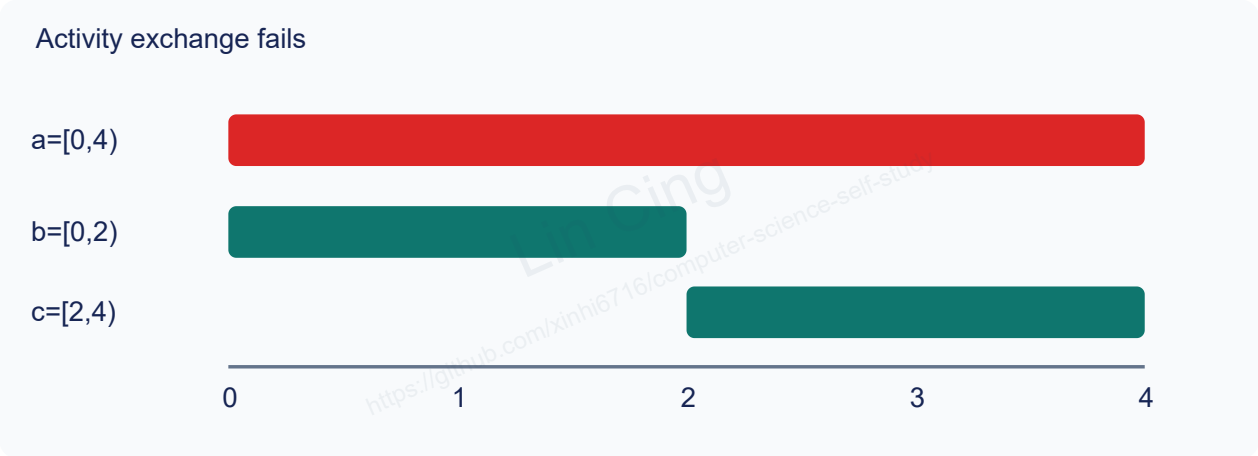
The uniform matroid $U_{2,3}$ has E={a,b,c} and $\mathcal{I}=\{X\subseteq E:|X|\leq 2\}$. Every two-element set is a basis. No singleton is maximal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Activity selection does not form a matroid

Use half-open intervals a=[0,4), b=[0,2), and c=[2,4). Independent sets contain pairwise compatible activities.

A={a} and B={b,c} are independent, with $|A|=1<2=|B|$. Neither $A\cup\{b\}$ nor $A\cup\{c\}$ is independent. Exchange fails.



Heredity still holds. The earliest-finish proof for activity selection does not make this independence system a matroid.

Weighted greedy and negative weights

Maximize $\Sigma_{x\in A} w(x)$ subject to $A\in\mathcal{I}$. Sort elements by decreasing weight. Start with $A=\varnothing$ and add x exactly when $A\cup\{x\}\in\mathcal{I}$.

For $w\geq 0$, this returns a maximum-weight independent set. Zero-weight elements may be added, and the result is a basis. Sorting costs O(n log n). With n independence queries of cost T_ind, total time is O(n log n+nT_ind).

For arbitrary weights and unrestricted independent sets, skip every negative element. In $U_{2,2}$ with weights 5 and −7, filling a basis yields −2 while selecting only the first yields 5. If the objective explicitly requires a basis, negative elements cannot be skipped. The original rule still finds a maximum-weight basis.

Exchange proof 1 | prepare an optimal basis

First assume nonnegative weights. Extend an optimal independent set to a basis without decreasing its weight. Thus an optimal basis O exists.

Maintain the invariant that some optimal basis O contains the current greedy set G. Initially $G=\varnothing$. Let g be the next accepted element. If $g\in O$, retain O.

Otherwise start with $I_0=G\cup\{g\}$ and set $H=I_0$. While $|H|<|O|$, exchange adds an element of $O\setminus H$ to H. Continue until $|H|=|O|=r$. Every step preserves independence and retains $G\cup\{g\}$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exchange proof 2 | replace one element without loss

Now $H\subseteq O\cup\{g\}$, $|H|=|O|$, and $G\cup\{g\}\subseteq H$. Therefore $H=(O\{y\})\cup\{g\}$ for some $y\in O\setminus G$.

Because $G\cup\{y\}\subseteq O$, y is feasible at the current G. It cannot have been rejected earlier. An earlier rejection would contradict heredity, since that earlier prefix is a subset of G. Thus decreasing weight order gives $w(g)\geq w(y)$.

$w(H)=w(O)-w(y)+w(g)\geq w(O)$. Optimality of O makes H optimal too, and H contains the new G.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exchange proof 2 | replace one element without loss (continued)

One replacement preserves the greedy prefix

$G \subseteq O$ → $I_0 = G \cup \{g\}$ → $H = O - \{y\} + \{g\}$

$I_0 \subseteq H \subseteq O \cup \{g\}$, $|H| = |O| = r$

Extend by elements of O, preserving independence

$y \notin G$, $w(g) \geq w(y)$, $w(H) \geq w(O)$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

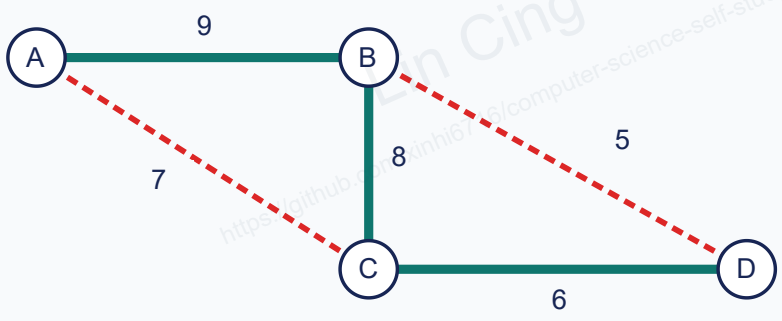
At termination G is a basis, so an optimal basis containing G equals G. For unrestricted sets with negative weights, apply this proof to the restriction to nonnegative elements.

Graphic matroid | feasible edge sets are forests

Take an undirected graph with V={A,B,C,D} and E={AB,BC,AC,CD,BD}. Let $\mathcal{I}$ contain its acyclic edge sets.

Deleting forest edges preserves acyclicity. If forests F,H satisfy |F|<|H|, some edge of H crosses two components of F. Otherwise H has at most $|V_i|-1$ edges inside each F component, totaling at most |F|. Adding a crossing edge to F creates no cycle. This proves exchange.

Accepted and rejected edges in a maximum-weight forest



AB, BC, CD

9 + 8 + 6 = 23

AC, BD rejected

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

This example maximizes forest weight. It does not develop a minimum spanning tree algorithm.

Graphic matroid | trace every decision

Use AB=9, BC=8, AC=7, CD=6, and BD=5.

Edge	Decision	G after scan	Weight
AB	Accept, joins A and B	{AB}	9
BC	Accept, joins C	{AB,BC}	17
AC	Reject, cycle A–B–C–A	{AB,BC}	17
CD	Accept, joins D	{AB,BC,CD}	23
BD	Reject, cycle B–C–D–B	{AB,BC,CD}	23

The three accepted edges form a basis of weight 23. The independence query asks whether adding an edge preserves a forest.

Unit jobs | minimize penalties by maximizing saved weight

Every job is available at time 0. One processor runs each job for exactly one time unit without preemption. Each integer deadline d_i lies in [1,n]. Completion C_i≤d_i incurs no penalty. Otherwise the nonnegative penalty p_i is paid. Every job must eventually run.

An independent set A is a job subset that can all finish on time. Define N_t(A)=|{i∈A:d_i≤t}|.

min Σ_late p_i = Σ_all p_i − max_(A∈$\mathcal{I}$) Σ_i∈A p_i.

Job	a_1	a_2	a_3	a_4	a_5	a_6	a_7
d_i	4	2	4	3	1	4	6
p_i	70	60	50	40	30	20	10

Data follow CLRS Figure 16.7. Total penalty is 280.

Feasibility and the schedule construction

$A \in \mathcal{I}$ iff $N_t(A) \leq t$ for every integer $t \in [0,n]$, iff increasing-deadline order completes every job on time.

Necessity. If $N_t > t$, more than t unit jobs must finish in the first t slots. This is impossible.

Sufficiency. Sort deadlines as $d_{(1)} \leq \ldots \leq d_{(k)}$. If job j is late, then $d_{(j)} < j$ and $N_{d_{(j)}} \geq j > d_{(j)}$, a contradiction. Hence job j finishes at time $j \leq d_{(j)}$.

Heredity follows from $N_t(B) \leq N_t(A)$ for $B \subseteq A$. Penalties determine the greedy scan order, not feasibility.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Job exchange | check every time prefix

Let A,B be feasible with $|A| < |B|$. Set $k = \max\{t : N_t(B) \leq N_t(A)\}$. This exists since both counts are zero at t=0. Since $N_n(B) > N_n(A)$, $k < n$.

The count difference is nonpositive at k and positive at k+1. Thus B has more jobs with deadline exactly k+1 than A. Choose $x \in B \setminus A$ with $d_x = k+1$.

- For $t \leq k$, $N_t(A \cup \{x\}) = N_t(A) \leq t$.
- For $t \geq k+1$, the integer count gap gives $N_t(A)+1 \leq N_t(B) \leq t$.

Therefore $A \cup \{x\}$ is feasible. Together with heredity and feasibility of $\varnothing$, this proves the deadline matroid.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Full selection trace | show the violated prefix

Scan by decreasing penalty. Each N vector lists t=1,…,7. Update A only after acceptance.

Candidate	Candidate $N_1,\ldots,N_7$	Decision	Saved penalty
a_1	(0,0,0,1,1,1,1)	Accept	70
a_2	(0,1,1,2,2,2,2)	Accept	130
a_3	(0,1,1,3,3,3,3)	Accept	180
a_4	(0,1,2,4,4,4,4)	Accept	220
a_5	(1,2,3,5,5,5,5)	Reject, $N_4=5>4$	220
a_6	(0,1,2,5,5,5,5)	Reject, $N_4=5>4$	220
a_7	(0,1,2,4,4,5,5)	Accept	230

The on-time set is $\{a_1,a_2,a_3,a_4,a_7\}$. Minimum total penalty is $280-230=50$.

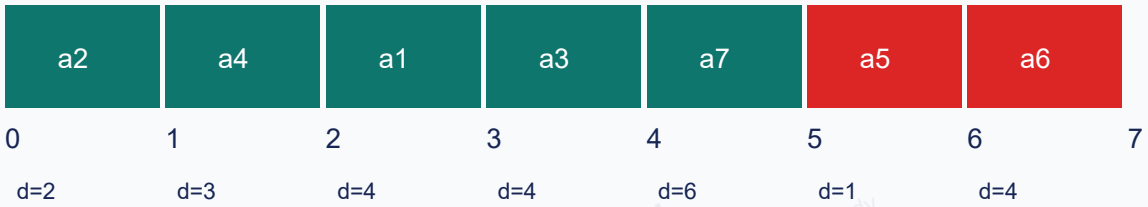
Full schedule | selection order is not execution order

Sort the accepted jobs by increasing deadline, then append rejected jobs. Break equal-deadline ties arbitrarily.

Slot	Job	Completion C_i	d_i	Status	Penalty
[0,1)	a_2	1	2	On time	0
[1,2)	a_4	2	3	On time	0
[2,3)	a_1	3	4	On time	0
[3,4)	a_3	4	4	On time	0
[4,5)	a_7	5	6	On time	0
[5,6)	a_5	6	1	Late	30
[6,7)	a_6	7	4	Late	20

Full schedule | selection order is not execution order (continued)

Schedule the on-time set before the late jobs



d=2 d=3 d=4 d=4 d=6 d=1 d=4

Σ saved = 230; Σ late penalty = 30 + 20 = 50

Different processing times, release times, or multiple processors require a new check of the feasibility axioms. This proof covers only the stated unit-job model.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 16, Greedy Algorithms. Original handwritten scanned algorithms notes (演算法_wjungle), greedy-method pages covering activity selection, Huffman coding, matroids, and scheduling. MIT 6.046J, Fall 2004, Lecture 13, Greedy Algorithms and Minimum Spanning Trees; Fall 2005, Lecture 16. These MIT lectures support the greedy and MST proof framework, not the original Huffman exposition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study